

CS 2201

Computer Algorithms

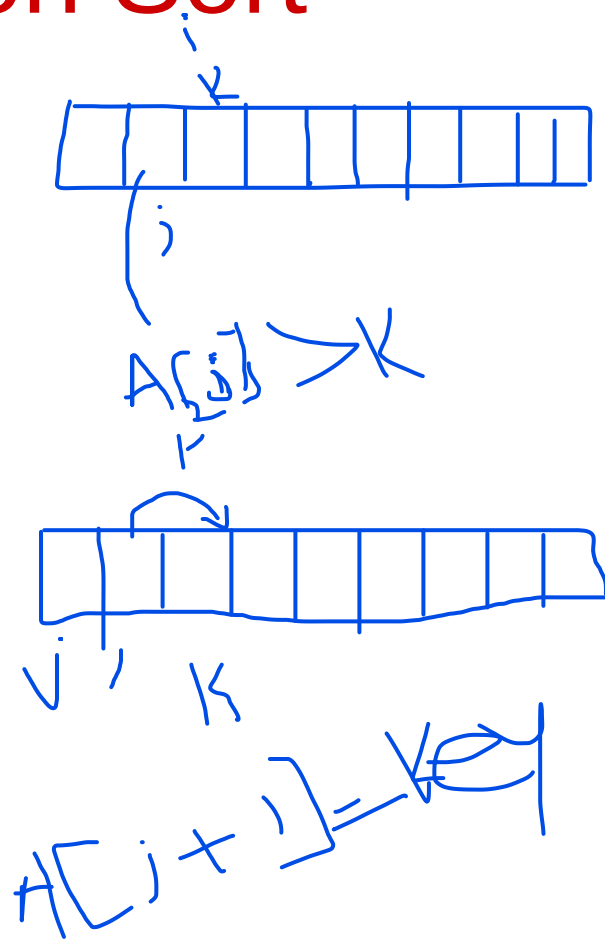
Asymptotic Performance

Review: Running Time

- Number of primitive steps that are executed
 - Except for time of executing a function call most statements roughly require the same amount of time
 - We can be more exact if need be

An Example: Insertion Sort

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

30	10	40	20
1	2	3	4

$i = \emptyset$	$j = \emptyset$	$\text{key} = \emptyset$
$A[j] = \emptyset$	$A[j+1] = \emptyset$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

30	10	40	20
1	2	3	4

$i = 2$	$j = 1$	$\text{key} = 10$
$A[j] = 30$	$A[j+1] = 10$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

30	30	40	20
1	2	3	4

$i = 2$	$j = 1$	$\text{key} = 10$
$A[j] = 30$	$A[j+1] = 30$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

30	30	40	20
1	2	3	4

$i = 2$	$j = 1$	$\text{key} = 10$
$A[j] = 30$	$A[j+1] = 30$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

30	30	40	20
1	2	3	4

$i = 2$	$j = 0$	$\text{key} = 10$
$A[j] = \emptyset$	$A[j+1] = 30$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

30	30	40	20
1	2	3	4

$i = 2$	$j = 0$	$\text{key} = 10$
$A[j] = \emptyset$	$A[j+1] = 30$	

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 2$	$j = 0$	$\text{key} = 10$
$A[j] = \emptyset$	$A[j+1] = 10$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$	$j = 0$	$\text{key} = 10$
$A[j] = \emptyset$	$A[j+1] = 10$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$	$j = 0$	$\text{key} = 40$
$A[j] = \emptyset$	$A[j+1] = 10$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$	$j = 0$	$\text{key} = 40$
$A[j] = \emptyset$	$A[j+1] = 10$	



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$	$j = 2$	$\text{key} = 40$
$A[j] = 30$	$A[j+1] = 40$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$	$j = 2$	$\text{key} = 40$
$A[j] = 30$	$A[j+1] = 40$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$	$j = 2$	$\text{key} = 40$
$A[j] = 30$	$A[j+1] = 40$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$	$j = 2$	$\text{key} = 40$
$A[j] = 30$	$A[j+1] = 40$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$	$j = 2$	$key = 20$
$A[j] = 30$	$A[j+1] = 40$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$	$j = 2$	$\text{key} = 20$
$A[j] = 30$	$A[j+1] = 40$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$	$j = 3$	$\text{key} = 20$
$A[j] = 40$	$A[j+1] = 20$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$	$j = 3$	$\text{key} = 20$
$A[j] = 40$	$A[j+1] = 20$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$	$j = 3$	$key = 20$
$A[j] = 40$	$A[j+1] = 40$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$	$j = 3$	$key = 20$
$A[j] = 40$	$A[j+1] = 40$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$	$j = 3$	$key = 20$
$A[j] = 40$	$A[j+1] = 40$	

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$	$j = 2$	$key = 20$
$A[j] = 30$	$A[j+1] = 40$	

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$	$j = 2$	$key = 20$
$A[j] = 30$	$A[j+1] = 40$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	30	40
1	2	3	4

$i = 4$	$j = 2$	$key = 20$
$A[j] = 30$	$A[j+1] = 30$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	30	40
1	2	3	4

$i = 4$	$j = 2$	$\text{key} = 20$
$A[j] = 30$	$A[j+1] = 30$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	30	40
1	2	3	4

$i = 4$	$j = 1$	$key = 20$
$A[j] = 10$	$A[j+1] = 30$	

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

10	30	30	40
1	2	3	4

$i = 4$	$j = 1$	$\text{key} = 20$
$A[j] = 10$	$A[j+1] = 30$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	20	30	40
1	2	3	4

$i = 4$	$j = 1$	$key = 20$
$A[j] = 10$	$A[j+1] = 20$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

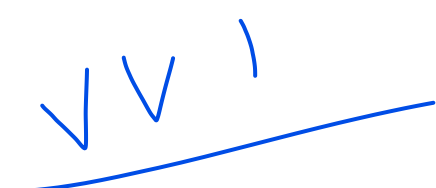
10	20	30	40
1	2	3	4

$i = 4$	$j = 1$	$\text{key} = 20$
$A[j] = 10$	$A[j+1] = 20$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

Done!

Insertion Sort



Statement	Effort
InsertionSort(A, n) {	
for i = 2 to n {	$c_1 n$
key = A[i]	$c_2(n-1)$
j = i - 1;	$c_3(n-1)$
while (j > 0) and (A[j] > key) {	$c_4 T$
A[j+1] = A[j]	$c_5(T-(n-1))$
j = j - 1	$c_6(T-(n-1))$
}	0
A[j+1] = key	$c_7(n-1)$
}	0
}	

$T = t_2 + t_3 + \dots + t_n$ where t_i is number of while expression evaluations for the i^{th} for loop iteration

Analyzing Insertion Sort

- $T(n) = c_1n + c_2(n-1) + c_3(n-1) + c_4T + c_5(T - (n-1)) + c_6(T - (n-1)) + c_7(n-1)$
 $= c_8T + c_9n + c_{10}$
- What can T be?
 - Best case -- inner loop body never executed
 - $t_i = 1 \rightarrow T(n)$ is a linear function
 - Worst case -- inner loop body executed for all previous elements
 - $t_i = i \rightarrow \underline{T(n) \text{ is a quadratic function}}$
 - Average case
 - ???

Types of analyses

Best case. Lower bound on cost.

- Determined by "easiest" input.
- Provides a goal for all inputs.

Worst case. Upper bound on cost.

- Determined by "most difficult" input.
- Provides a guarantee for all inputs.

Average case. Expected cost for random input.

- Need a model for "random" input.
- Provides a way to predict performance.

Analysis

- Simplifications
 - Ignore actual and abstract statement costs
 - Order of growth is the interesting measure:
 - Highest-order term is what counts
 - ⌘ Remember, we are doing asymptotic analysis
 - ⌘ As the input size grows larger it is the high order term that dominates

Upper Bound Notation

- We say InsertionSort's run time is $O(n^2)$
 - Properly we should say run time is *in* $O(n^2)$
 - Read O as “Big-O” (you’ll also hear it as “order”)
- In general a function
 - $T(n)$ is $O(f(n))$ if there exist positive constants c and n_0 such that $T(n) \leq c \cdot f(n)$ for all $n \geq n_0$
- Formally
 - $O(f(n)) = \{ T(n) : \exists \text{ positive constants } c \text{ and } n_0 \text{ such that } T(n) \leq c \cdot f(n) \forall n \geq n_0 \}$

Big-Oh:English

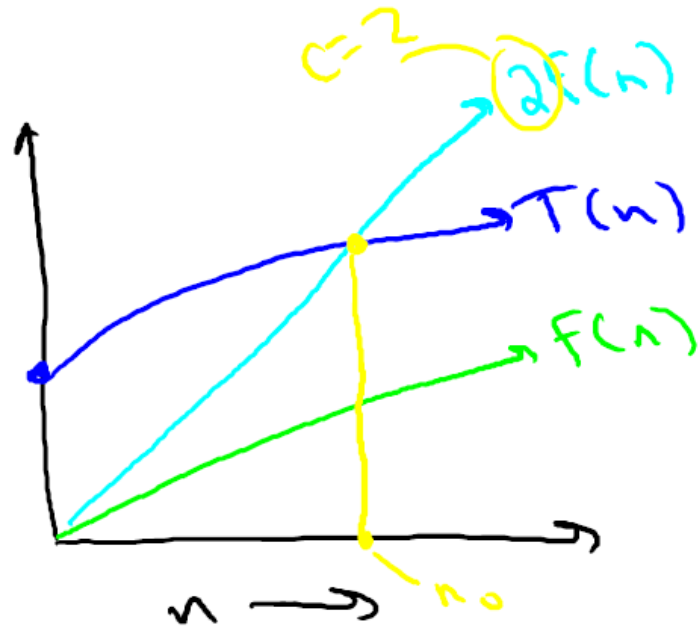
Let $T(n)$ = function on $n = 1, 2, 3, \dots$

[usually, the worst-case running time of an algorithm]

Q : When is $T(n) = O(f(n))$?

A : if eventually (for all sufficiently large n), $T(n)$ is bounded above by a constant multiple of $f(n)$

Big-Oh: Formal



Picture $T(n) = O(f(n))$

Formal Definition : $T(n) = O(f(n))$ if
and only if there exist constants

$c, n_0 > 0$ such that

$$T(n) \leq c \cdot f(n)$$

For all $n \geq n_0$

Warning : c, n_0 cannot depend on n

Example 1

Claim : if $T(n) = a_k n^k + \dots + a_1 n + a_0$ then

$$T(n) = O(n^k)$$

Proof : Choose $n_0 = 1$ and $c = |a_k| + |a_{k-1}| + \dots + |a_1| + |a_0|$

Need to show that $\forall n \geq 1, T(n) \leq c \cdot n^k$

We have, for every $n \geq 1$,

$$\begin{aligned} T(n) &\leq |a_k|n^k + \dots + |a_1|n + |a_0| \\ &\leq |a_k|n^k + \dots + |a_1|n^k + |a_0|n^k \\ &= c \cdot n^k \end{aligned}$$

Example 2

Claim : for every $k \geq 1$, n^k is not $O(n^{k-1})$

Proof : by contradiction. Suppose $n^k = O(n^{k-1})$

Then there exist constants c, n_0 such that

$$n^k \leq c \cdot n^{k-1} \quad \forall n \geq n_0$$

But then [cancelling n^{k-1} from both sides]:

$$n \leq c \quad \forall n \geq n_0$$

Which is clearly False [contradiction].

Insertion Sort Is $O(n^2)$

- Proof

- Suppose runtime is $an^2 + bn + c$

- If any of a , b , and c are less than 0 replace the constant with its absolute value

$$\begin{aligned} an^2 + bn + c &\leq (a + b + c)n^2 + (a + b + c)n + (a + b + c) \\ &\leq 3(a + b + c)n^2 \text{ for } n \geq 1 \end{aligned}$$

- Let $c' = 3(a + b + c)$ and let $n_0 = 1$

- Question

- Is InsertionSort $O(n^3)$?
- Is InsertionSort $O(n)$?

Types of analyses

Best case. Lower bound on cost.

Worst case. Upper bound on cost.

Average case. “Expected” cost.

Actual data might not match input model?

- Need to understand input to effectively process it.
- Approach 1: design for the worst case.
- Approach 2: randomize, depend on probabilistic guarantee.

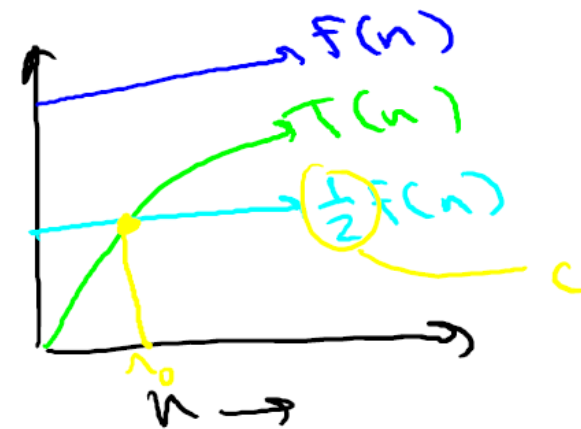
Lower Bound Notation

Definition : $T(n) = \Omega(f(n))$

If and only if there exist constants c, n_0 such that

$$T(n) \geq c \cdot f(n) \quad \forall n \geq n_0.$$

Picture



$$T(n) = \Omega(f(n))$$

Lower Bound Notation

- We say InsertionSort's run time is $\Omega(n)$
- In general a function
 - $T(n)$ is $\Omega(f(n))$ if $\exists$ positive constants c and n_0 such that $0 \leq c \cdot f(n) \leq T(n) \quad \forall n \geq n_0$
- Proof:
 - Suppose run time is $an + b$
 - Assume a and b are positive
 - $an \leq an + b$

Asymptotic Tight Bound

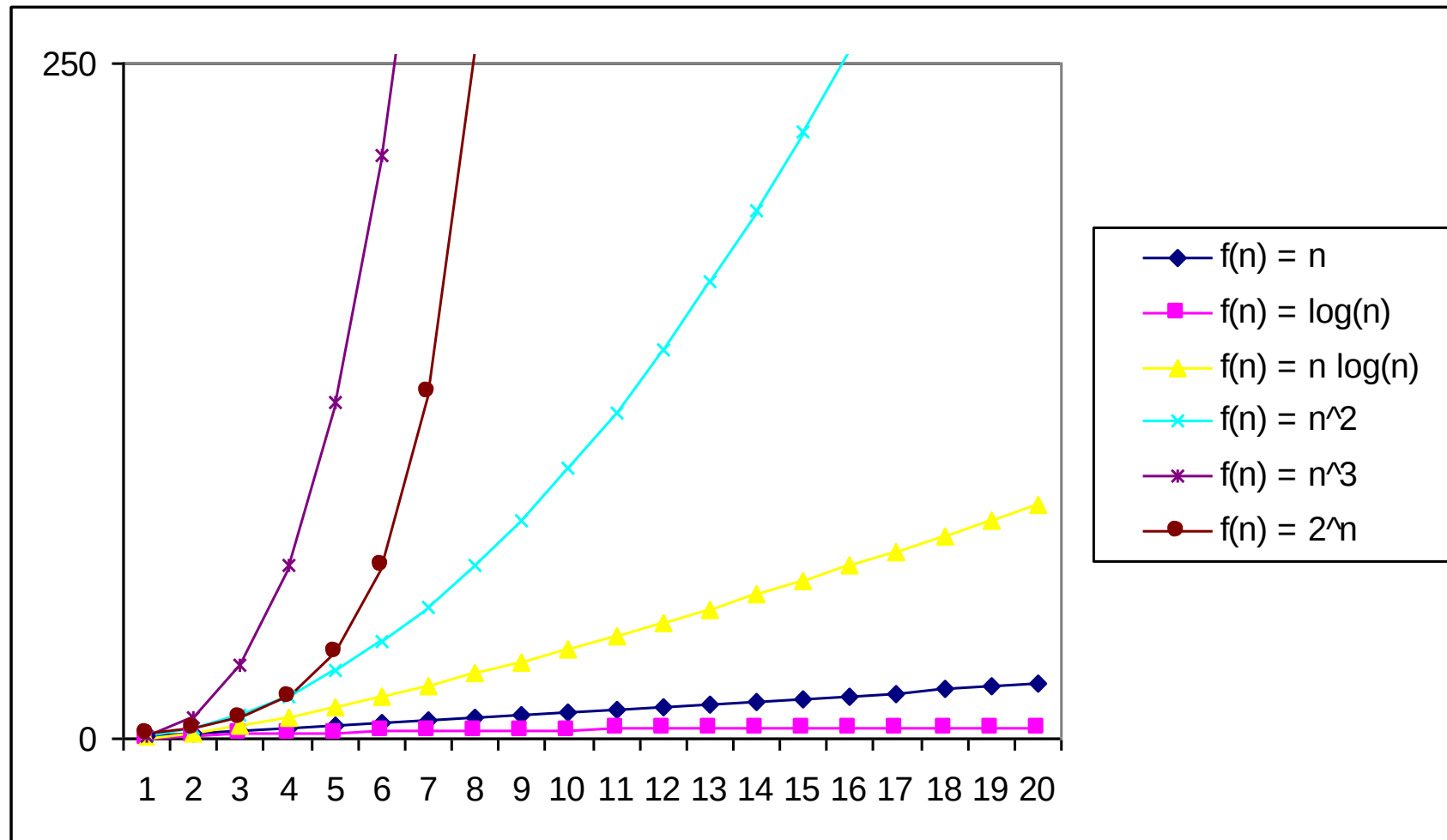
Definition : $T(n) = \theta(f(n))$ if and only if
 $T(n) = O(f(n))$ and $T(n) = \Omega(f(n))$

Equivalent : there exist constants c_1, c_2, n_0 such that

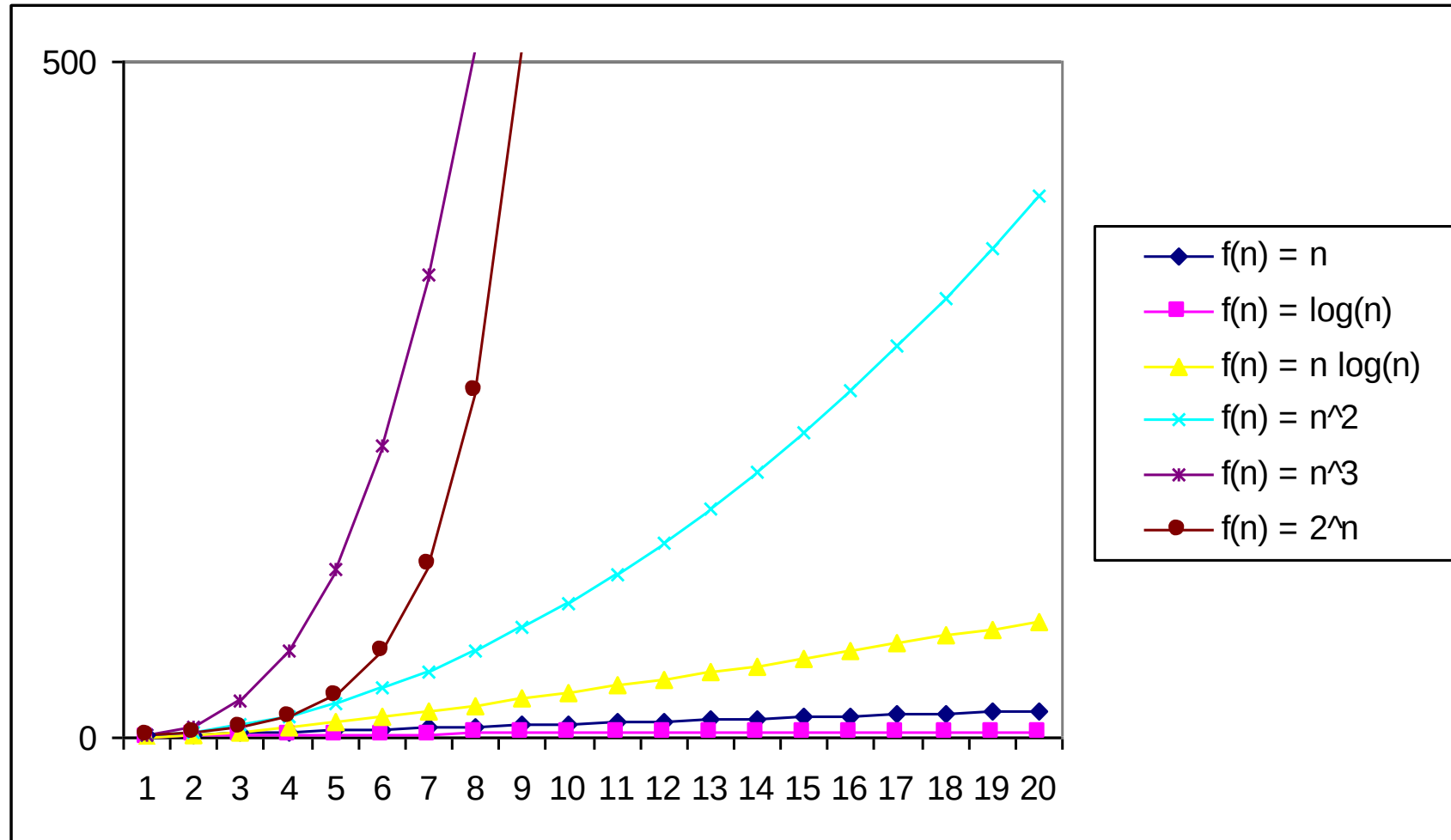
$$c_1 f(n) \leq T(n) \leq c_2 f(n)$$

$$\forall n \geq n_0$$

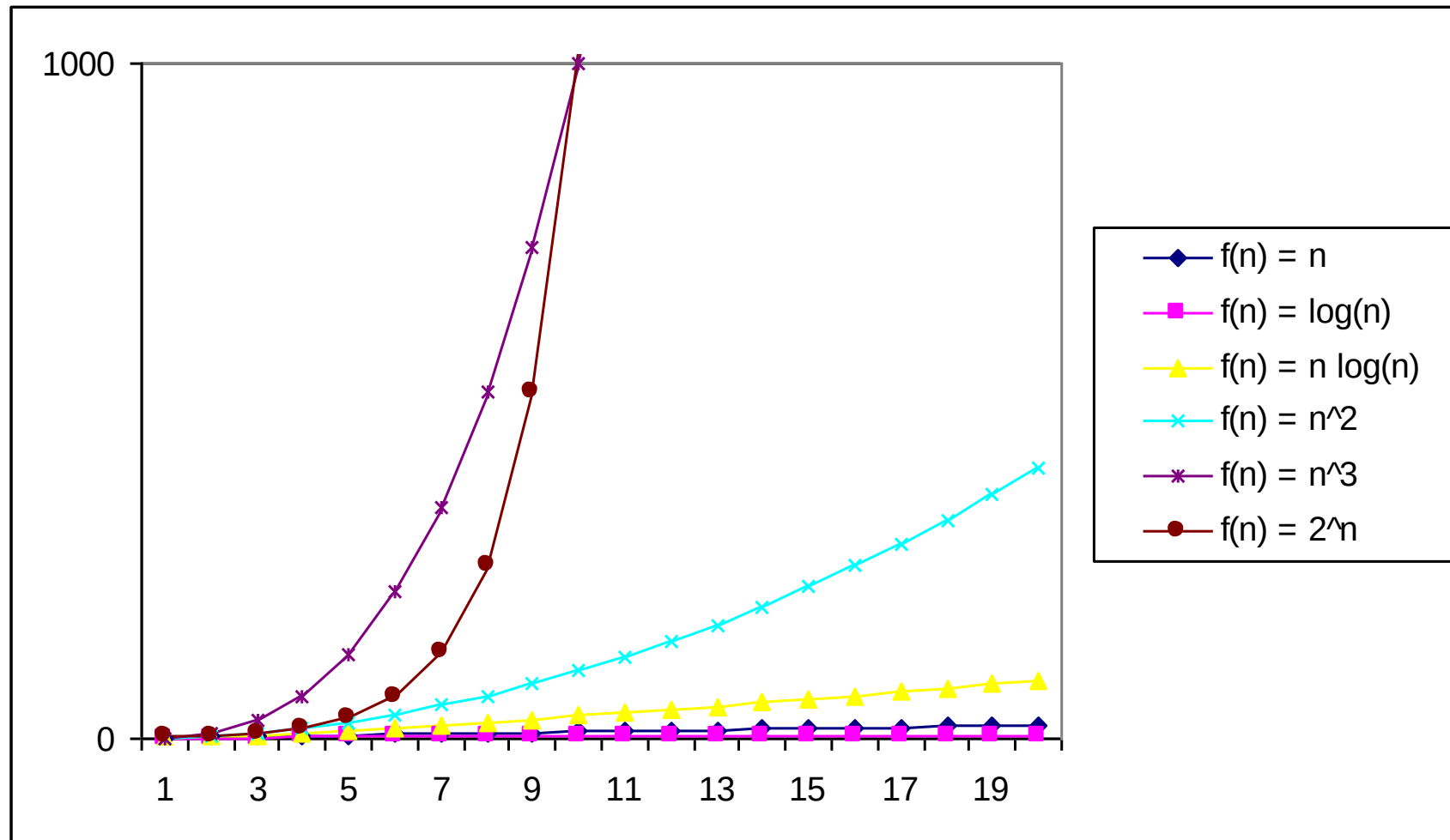
Practical Complexity



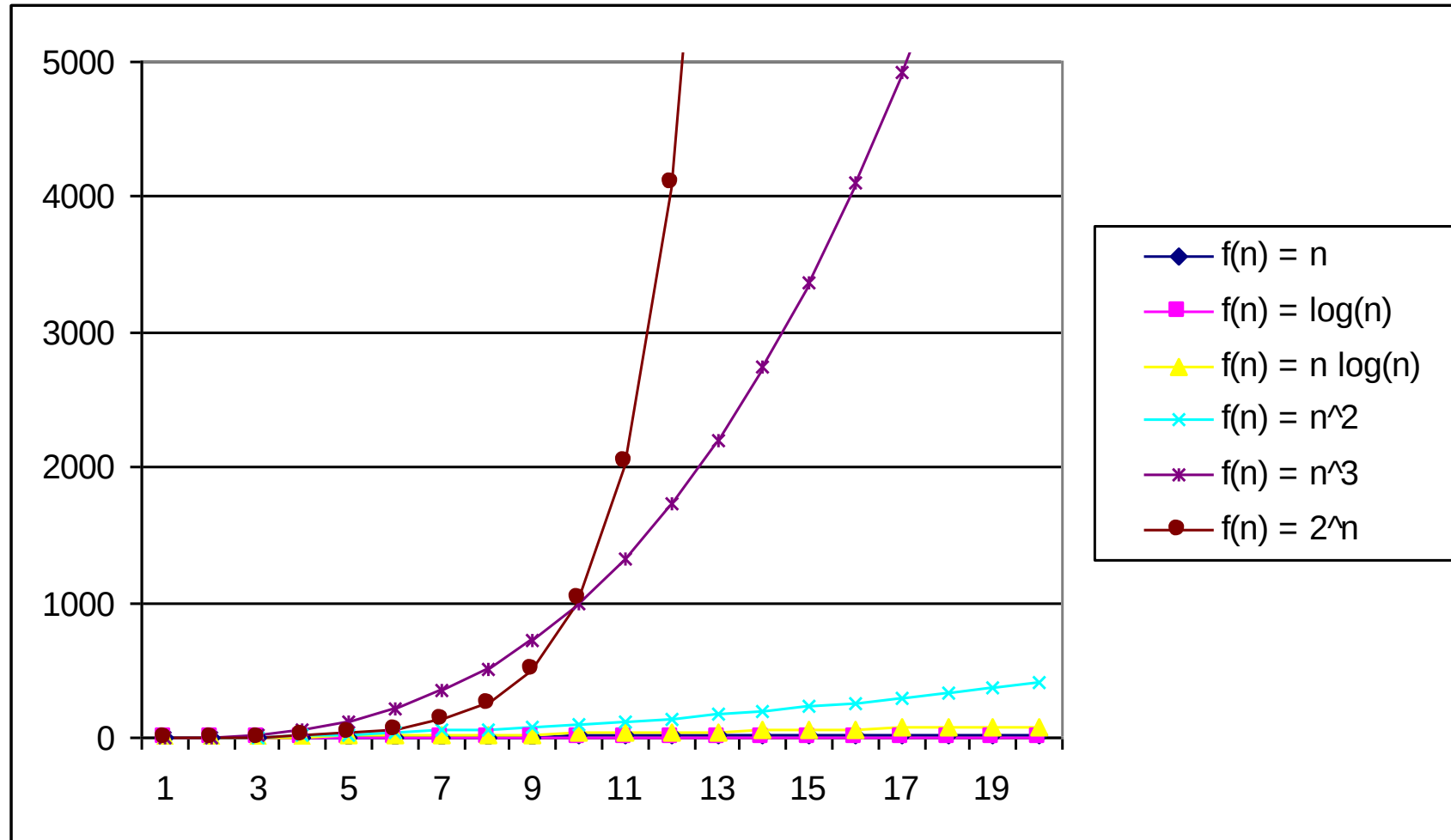
Practical Complexity



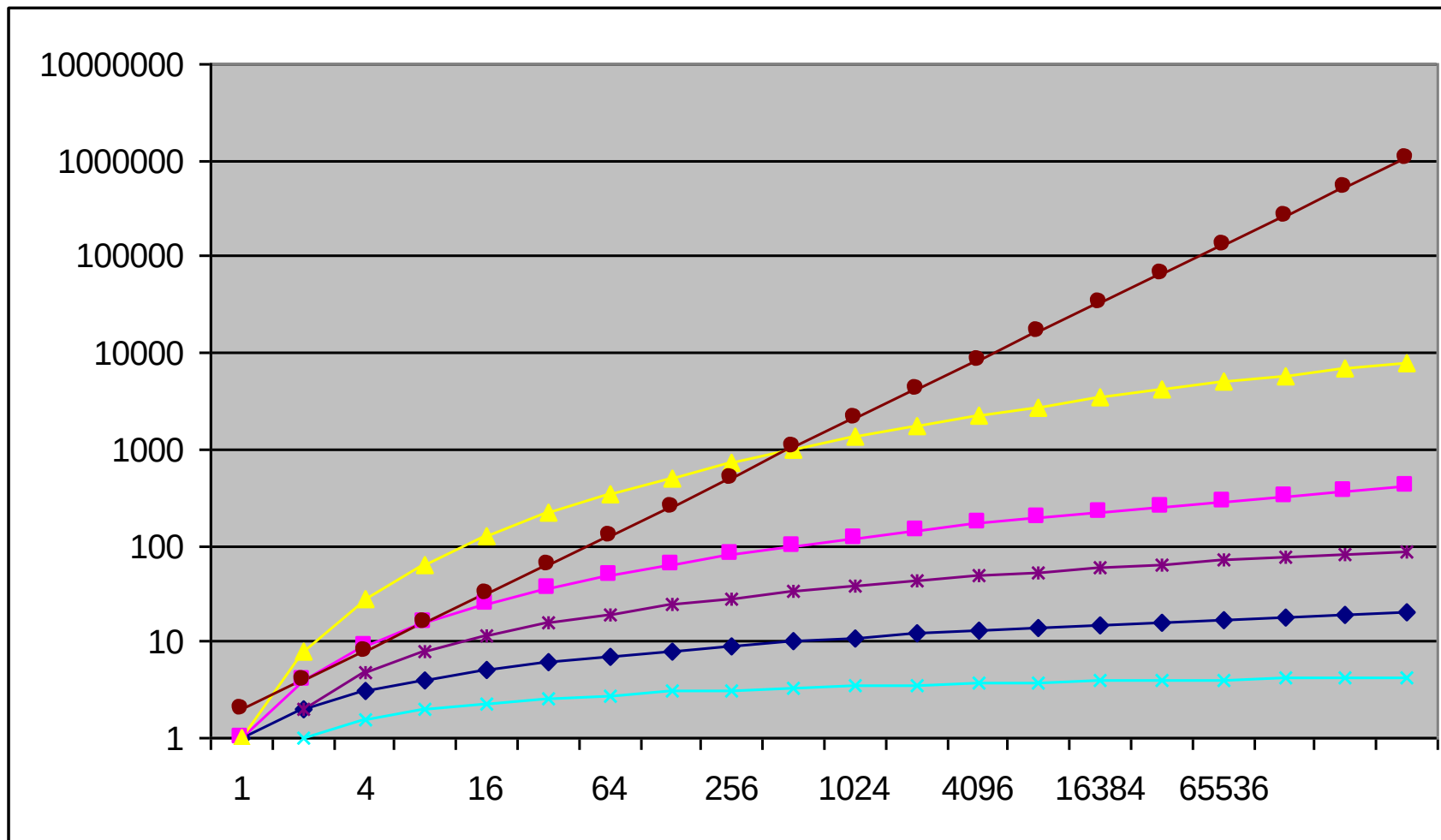
Practical Complexity



Practical Complexity



Practical Complexity



Other Asymptotic Notations

- A function $f(n)$ is $o(g(n))$ if $\exists$ positive constants c and n_0 such that

$$f(n) < c g(n) \quad \forall n \geq n_0$$

- A function $f(n)$ is $\omega(g(n))$ if $\exists$ positive constants c and n_0 such that

$$c g(n) < f(n) \quad \forall n \geq n_0$$

- Intuitively,
 - $o()$ is like $<$
 - $\omega()$ is like $>$
 - $\Theta()$ is like $=$
 - $O()$ is like $\leq$
 - $\Omega()$ is like $\geq$

Theory of algorithms

Goals.

- Establish “difficulty” of a problem.
- Develop “optimal” algorithms.

Approach.

- Suppress details in analysis: analyze “to within a constant factor”.
- Eliminate variability in input model by focusing on the worst case.

Optimal algorithm.

- Performance guarantee (to within a constant factor) for any input.
- No algorithm can provide a better performance guarantee.

Commonly-used notations in the theory of algorithms

notation	provides	example	shorthand for	used to
Big Theta	asymptotic order of growth	$\Theta(N^2)$	$\frac{1}{2} N^2$ $10 N^2$ $5 N^2 + 22 N \log N + 3 N$ $\vdots$	classify algorithms
Big Oh	$\Theta(N^2)$ and smaller	$O(N^2)$	$10 N^2$ $100 N$ $22 N \log N + 3 N$ $\vdots$	develop upper bounds
Big Omega	$\Theta(N^2)$ and larger	$\Omega(N^2)$	$\frac{1}{2} N^2$ N^5 $N^3 + 22 N \log N + 3 N$ $\vdots$	develop lower bounds

Theory of algorithms: example 1

Goals.

- Establish “difficulty” of a problem and develop “optimal” algorithms.
- Ex. 1-SUM = “*Is there a 0 in the array?*”

Upper bound. A specific algorithm.

- Ex. Brute-force algorithm for 1-SUM: Look at every array entry.
- Running time of the optimal algorithm for 1-SUM is $O(N)$.

Lower bound. Proof that no algorithm can do better.

- Ex. Have to examine all N entries (any unexamined one might be 0).
- Running time of the optimal algorithm for 1-SUM is $\Omega(N)$.

Optimal algorithm.

- Lower bound equals upper bound (to within a constant factor).
- Ex. Brute-force algorithm for 1-SUM is optimal: its running time is $\Theta(N)$.

Goals.

- Establish “difficulty” of a problem and develop “optimal” algorithms.
- Ex. 3-SUM.

Upper bound. A specific algorithm.

- Ex. Improved algorithm for 3-SUM.
- Running time of the optimal algorithm for 3-SUM is $O(N^2 \log N)$.

Lower bound. Proof that no algorithm can do better.

- Ex. Have to examine all N entries to solve 3-SUM.
- Running time of the optimal algorithm for solving 3-SUM is $\Omega(N)$.

Open problems.

- Optimal algorithm for 3-SUM?
- Subquadratic algorithm for 3-SUM?
- Quadratic lower bound for 3-SUM?

Where Does Notation Come From?

“On the basis of the issues discussed here, I propose that members of SIGACT, and editors of computer science and mathematics journals, adopt the O , Ω , and Θ notations as defined above, unless a better alternative can be found reasonably soon”.

-D. E. Knuth, “Big Omicron and Big Omega and Big Theta”, SIGACT News, 1976. Reprinted in “Selected Papers on Analysis of Algorithms.”

The End